Use CI/CD configuration from other files

Tier: Free, Premium, Ultimate

Offering: GitLab.com, GitLab Self-Managed, GitLab Dedicated

You can use `include` to include external YAML files in your CI/CD jobs.

Include a single configuration file

To include a single configuration file, use `include` by itself with a single file with either of these syntax options:

- On the same line:

```yaml
include: 'my-config.yml'
```

- As a single item in an array:

```yaml
include:
  - 'my-config.yml'
```

If the file is a local file, the behavior is the same as `include:local`. If the file is a remote file, it is the same as `include:remote`.

Include an array of configuration files

You can include an array of configuration files:

- If you do not specify an `include` type, each array item defaults to `include:local` or `include:remote`, as needed:

```yaml
include:
  - 'https://gitlab.com/awesome-project/raw/main/.before-script-template.yml'
  - 'templates/.after-script-template.yml'
```

- You can define a single item array:

```yaml
include:
  - remote: 'https://gitlab.com/awesome-project/raw/main/.before-script-template.yml'
```

- You can define an array and explicitly specify multiple `include` types:

```yaml
include:
  - remote: 'https://gitlab.com/awesome-project/raw/main/.before-script-template.yml'
  - local: 'templates/.after-script-template.yml'
  - template: Auto-DevOps.gitlab-ci.yml
```

```yaml
include:
  - 'https://gitlab.com/awesome-project/raw/main/.before-script-template.yml'
  - 'templates/.after-script-template.yml'
  - template: Auto-DevOps.gitlab-ci.yml
  - project: 'my-group/my-project'
    ref: main
    file: 'templates/.gitlab-ci-template.yml'
```

Use `default` configuration from an included configuration file

You can define a `default` section in a configuration file. When you use a `default` section with the `include` keyword, the defaults apply to all jobs in the pipeline.

For example, you can use a `default` section with `before_script`.

Content of a custom configuration file named `/templates/.before-script-template.yml`:

```yaml
default:
  before_script:
    - apt-get update -qq && apt-get install -y -qq sqlite3 libsqlite3-dev nodejs
    - gem install bundler --no-document
    - bundle install --jobs $(nproc)  "${FLAGS[@]}"
```

Content of `.gitlab-ci.yml`:

```yaml
include: 'templates/.before-script-template.yml'

rspec1:
  script:
    - bundle exec rspec

rspec2:
  script:
    - bundle exec rspec
```

The default `before_script` commands execute in both `rspec` jobs, before the `script` commands.

Override included configuration values

When you use the `include` keyword, you can override the included configuration values to adapt them to your pipeline requirements.

The following example shows an `include` file that is customized in the `.gitlab-ci.yml` file. Specific YAML-defined variables and details of the `production` job are overridden.

Content of a custom configuration file named `autodevops-template.yml`:

```yaml
```

```
  POSTGRES_DB: $CI_ENVIRONMENT_SLUG

production:
  stage: production
  script:
    - install_dependencies
    - deploy
  environment:
    name: production
    url: https://$CI_PROJECT_PATH_SLUG.$KUBE_INGRESS_BASE_DOMAIN
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

Content of `.gitlab-ci.yml`:

```yaml
include: 'https://company.com/autodevops-template.yml'

default:
  image: alpine:latest

variables:
  POSTGRES_USER: root
  POSTGRES_PASSWORD: secure_password

stages:
  - build
  - test
  - production

production:
  environment:
    url: https://domain.com
```

The `POSTGRES_USER` and `POSTGRES_PASSWORD` variables and the `environment:url` of the `production` job defined in the `.gitlab-ci.yml` file override the values defined in the `autodevops-template.yml` file. The other keywords do not change. This method is called *merging*.

Merge method for `include`

The `include` configuration merges with the main configuration file with this process:

- Included files are read in the order defined in the configuration file, and the included configuration is merged together in the same order.
- If an included file also uses `include`, that nested `include` configuration is merged first (recursively).
- If parameters overlap, the last included file takes precedence when merging the configuration from the included files.
- After all configuration added with `include` is merged together, the main configuration is merged with the included configuration.

This merge method is a *deep merge*, where hash maps are merged at any depth in the configuration. To merge hash map “A” (that contains the configuration merged so far) and “B” (the next piece of configuration), the keys and values are processed as follows:

- When the key only exists in A, use the key and value from A.
- When the key exists in both A and B, and their values are both hash maps, merge those hash maps.
- When the key exists in both A and B, and one of the values is not a hash map, use the value from B.
- Otherwise, use the key and value from B.

```yaml
include: 'common.yml'

variables:
  POSTGRES_USER: username

test:
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
      when: manual
  artifacts:
    reports:
      junit: rspec.xml
```

- The `common.yml` file:

```yaml
variables:
  POSTGRES_USER: common_username
  POSTGRES_PASSWORD: testing_password

test:
  rules:
    - when: never
  script:
    - echo LOGIN=${POSTGRES_USER} > deploy.env
    - rake spec
  artifacts:
    reports:
      dotenv: deploy.env
```

The merged result is:

```yaml
variables:
  POSTGRES_USER: username
  POSTGRES_PASSWORD: testing_password

test:
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
      when: manual
  script:
    - echo LOGIN=${POSTGRES_USER} > deploy.env
    - rake spec
  artifacts:
    reports:
      junit: rspec.xml
      dotenv: deploy.env
```

In this example:

- Variables are only evaluated after all the files are merged together. A job in an included file might end up using a variable value defined in a different file.
- `rules` is an array so it cannot be merged. The top-level file takes precedence.
- `artifacts` is a hash map so it can be deep merged.

You can use merging to extend and override configuration in an included template, but you cannot add or modify individual items in an array. For example, to add an additional `notify_owner` command to the extended `production` job's `script` array:

Content of `autodevops-template.yml`:

```yaml
production:
  stage: production
  script:
    - install_dependencies
    - deploy
```

Content of `.gitlab-ci.yml`:

```yaml
include: 'autodevops-template.yml'

stages:
  - production

production:
  script:
    - install_dependencies
    - deploy
    - notify_owner
```

If `install_dependencies` and `deploy` are not repeated in the `.gitlab-ci.yml` file, the `production` job would have only `notify_owner` in the script.

Use nested includes

You can nest `include` sections in configuration files that are then included in another configuration. For example, for `include` keywords nested three deep:

Content of `.gitlab-ci.yml`:

```yaml
include:
  - local: /.gitlab-ci/another-config.yml
```

Content of `/.gitlab-ci/another-config.yml`:

```yaml
include:
  - local: /.gitlab-ci/config-defaults.yml
```

Content of `/.gitlab-ci/config-defaults.yml`:

```yaml
default:
  after_script:
    - echo "Job complete."
```

If any file changes the included configuration using overrides, then the order of the `include` entries might affect the final configuration. The last time the configuration is included overrides any previous times the file was included. For example:

- Contents of a `defaults.gitlab-ci.yml` file:

```yaml
default:
  before_script: echo "Default before script"
```

- Contents of a `unit-tests.gitlab-ci.yml` file:

```yaml
include:
  - template: defaults.gitlab-ci.yml

default:  # Override the included default
  before_script: echo "Unit test default override"

unit-test-job:
  script: unit-test.sh
```

- Contents of a `smoke-tests.gitlab-ci.yml` file:

```yaml
include:
  - template: defaults.gitlab-ci.yml

default:  # Override the included default
  before_script: echo "Smoke test default override"

smoke-test-job:
  script: smoke-test.sh
```

With these three files, the order they are included changes the final configuration. With:

- `unit-tests` included first, the contents of the `.gitlab-ci.yml` file is:

```yaml
include:
  - local: unit-tests.gitlab-ci.yml
  - local: smoke-tests.gitlab-ci.yml
```

The final configuration would be:

```yaml
unit-test-job:
 before_script: echo "Smoke test default override"
 script: unit-test.sh

smoke-test-job:
 before_script: echo "Smoke test default override"
 script: smoke-test.sh
```

- `unit-tests` included last, the contents of the `.gitlab-ci.yml` file is:

```yaml
```

- The final configuration would be:

```yaml
unit-test-job:
 before_script: echo "Unit test default override"
 script: unit-test.sh

smoke-test-job:
 before_script: echo "Unit test default override"
 script: smoke-test.sh
```

If no file overrides the included configuration, the order of the `include` entries does not affect the final configuration

Use variables with `include`

In `include` sections in your `.gitlab-ci.yml` file, you can use:

- Project variables.
- Group variables.
- Instance variables.
- Project predefined variables (`CI_PROJECT_*`).
- Trigger variables.
- Scheduled pipeline variables.
- Manual pipeline run variables.
- The `CI_PIPELINE_SOURCE` and `CI_PIPELINE_TRIGGERED` predefined variables.
- The `$CI_COMMIT_REF_NAME` predefined variable.

For example:

```yaml
include:
  project: '$CI_PROJECT_PATH'
  file: '.compliance-gitlab-ci.yml'
```

You cannot use variables defined in jobs, or in a global `variables` section which defines the default variables for all jobs. Includes are evaluated before jobs, so these variables cannot be used with `include`.

For an example of how you can include predefined variables, and the variables' impact on CI/CD jobs, see this CI/CD variable demo.

You cannot use CI/CD variables in an `include` section in a dynamic child pipeline's configuration. Issue 378717 proposes fixing this issue.

Use `rules` with `include`

History >

You can use `rules` with `include` to conditionally include other configuration files.

You can only use `rules` with certain variables, and these keywords:

- `rules:if`.

`include` with `rules:if`

History >

Use `rules:if` to conditionally include other configuration files based on the status of CI/CD variables. For example:

```yaml
include:
  - local: builds.yml
    rules:
      - if: $DONT_INCLUDE_BUILDS == "true"
        when: never
  - local: builds.yml
    rules:
      - if: $ALWAYS_INCLUDE_BUILDS == "true"
        when: always
  - local: builds.yml
    rules:
      - if: $INCLUDE_BUILDS == "true"
  - local: deploys.yml
    rules:
      - if: $CI_COMMIT_BRANCH == "main"

test:
  stage: test
  script: exit 0
```

`include` with `rules:exists`

History >

Use `rules:exists` to conditionally include other configuration files based on the existence of files. For example:

```yaml
include:
  - local: builds.yml
    rules:
      - exists:
          - exception-file.md
        when: never
  - local: builds.yml
    rules:
      - exists:
          - important-file.md
        when: always
  - local: builds.yml
    rules:
      - exists:
          - file.md

test:
  stage: test
  script: exit 0
```

In this example, GitLab checks for the existence of `file.md` in the current project.

```yaml
# Pipeline configuration in my-group/my-project
include:
  - project: my-group/other-project
    ref: other_branch
    file: other-file.yml

test:
  script: exit 0

# other-file.yml in my-group/other-project on ref other_branch
include:
  - project: my-group/my-project
    ref: main
    file: my-file.yml
    rules:
      - exists:
          - file.md
```

In this example, GitLab searches for the existence of `file.md` in `my-group/other-project` on commit ref `other_branch`, not the project/ref in which the pipeline runs.

To change the search context you can use `rules:exists:paths` with `rules:exists:project`. For example:

```yaml
include:
  - project: my-group/my-project
    ref: main
    file: my-file.yml
    rules:
      - exists:
          paths:
            - file.md
          project: my-group/my-project
          ref: main
```

`include` with `rules:changes`

History >

Use `rules:changes` to conditionally include other configuration files based on changed files. For example:

```yaml
```

```
      - changes:
        - Dockerfile
  - local: builds2.yml
    rules:
      - changes:
          paths:
            - Dockerfile
          compare_to: 'refs/heads/branch1'
        when: always
  - local: builds3.yml
    rules:
      - if: $CI_PIPELINE_SOURCE == "merge_request_event"
        changes:
          paths:
            - Dockerfile

test:
  stage: test
  script: exit 0
```

In this example:

- `builds1.yml` is included when `Dockerfile` has changed.
- `builds2.yml` is included when `Dockerfile` has changed relative to `refs/heads/branch1`.
- `builds3.yml` is included when `Dockerfile` has changed and the pipeline source is a merge request event. The jobs in `builds3.yml` must also be configured to run for merge request pipelines.

Use `include:local` with wildcard file paths

You can use wildcard paths (`*` and `**`) with `include:local`.

Example:

```yaml
include: 'configs/*.yml'
```

When the pipeline runs, GitLab:

- Adds all `.yml` files in the `configs` directory into the pipeline configuration.
- Does not add `.yml` files in subfolders of the `configs` directory. To allow this, add the following configuration:

```yaml
# This matches all `.yml` files in `configs` and any subfolder in it.
include: 'configs/**.yml'

# This matches all `.yml` files only in subfolders of `configs`.
include: 'configs/**/*.yml'
```

Troubleshooting

`Maximum of 150 nested includes are allowed!` error

The maximum number of nested included files for a pipeline is 150. If you receive the `Maximum 150 includes are allowed` error message in your pipeline, it's likely that either:

- Some of the nested configuration includes an overly large number of additional nested `include` configuration.

To help reduce the risk of this happening, edit the pipeline configuration file with the pipeline editor, which validates if the limit is reached. You can remove one included file at a time to try to narrow down which configuration file is the source of the loop or excessive included files.

In GitLab 16.0 and later users on GitLab Self-Managed can change the maximum includes value.

Error: `Local file <file> does not exist!` with `include:local`

You might receive a `Local file <file> does not exist!` error when using `include:local`, even though the file exists in the repository.

This error is a known system-level issue, not a CI/CD configuration problem. It has been observed intermittently in distributed Gitaly or Praefect setups. If you encounter this error, retry the pipeline.

BY-SA